digithings · edition 2026 · version 1.0

DIGITHINGS

the digithings book

CONTENTS

01 Why this book exists 3

02 Why self-hosted 4

03 How it scales 5

04 Pricing 6

05 digigraph, the orchestration hub 7

06 digillm, one model layer for every service 9

07 digichat, the interface people touch 11

08 digiquant, the research vertical 12

09 digivoice, the voice loop on your machine 14

10 digisearch, finding what matters 16

11 The Cloudflare stack and its workers 18

12 The local stack, one machine 20

13 MCP, every capability as a tool 22

14 Your machine first 24

01

Why this book exists

Most of what gets written about AI infrastructure is written by whoever is renting it to you. This book is written from the other side of that relationship: what changes when you run the thing yourself, and why self-hosting is a position worth choosing rather than an inconvenience to tolerate until you can afford otherwise.

It is not a tour of digithings' internals, the docs on this site already cover that, endpoint by endpoint. This is shorter, and argues a position about infrastructure in general: why it matters, how it scales, and what it costs. Where it needs an example, it reaches for the nearest one, which happens to be this project, but the argument is not about digithings.

02

Why self-hosted

Every rented service is a bet that the convenience stays worth the price, and that the terms do not change while you are not looking. AI infrastructure is no different, except that the thing being rented, compute, a model endpoint, a place to put your data, is unusually central to whatever you are building on top of it.

The other option is to run the stack yourself. Not as a hobby or a protest, but as a position with ordinary costs and an unusual upside.

03

How it scales

"Does it scale?" usually means one thing when asked of a rented service and a different thing when asked of a stack you run yourself. A rented service scales by raising a limit on someone else's dashboard, and the bill follows. A self-hosted stack scales by you deciding what to add, when, and that decision is yours to make badly or well.

The second shape is what grows, what can stay fixed, and where the ceiling sits once renting is no longer the constraint.

04

Pricing

"What does it cost?" is the question every argument in this book eventually has to answer, because an argument about ownership that ignores price is not a serious one.

The honest answer for self-hosted AI infrastructure has two parts: the software itself, and everything the software touches. They are priced differently, and conflating them is how "free" ends up meaning something misleading in either direction.

05

digigraph, the orchestration hub

Every system with more than one capability eventually needs someone to answer a hard question: when a request arrives, who decides what happens next? digigraph is digithings' answer. It is the orchestration hub every user request flows through, and its one design commitment is to stay thin. It routes, it dispatches, it streams, and it leaves the real work to the verticals that own it.

What it is for

A chat message, a research job and a dashboard widget all need the same things: understand the request, pick the right tools, run them in the right order, and return the result. Without a hub, each of those callers re-implements that loop, and they drift apart within months. digigraph owns the loop once, so digichat, the dashboard and outside callers all share one routing behaviour, one tool contract and one streaming shape.

The audience question this chapter answers is simple: where does the intelligence of the system live? Answer: in one place, on purpose.

How it works

digigraph runs a state machine. A request enters, the machine walks it through stages, research, validation, execution, and optional refinement, and conditional edges decide which stages run for each request. Around that machine sits a registry of named tools. Some tools live inside the hub; vertical services publish their own tool descriptions, which the hub fetches lazily and invokes over the network. The hub never carries a hard-coded list of what the verticals can do. It asks, at runtime, and dispatches what it finds.

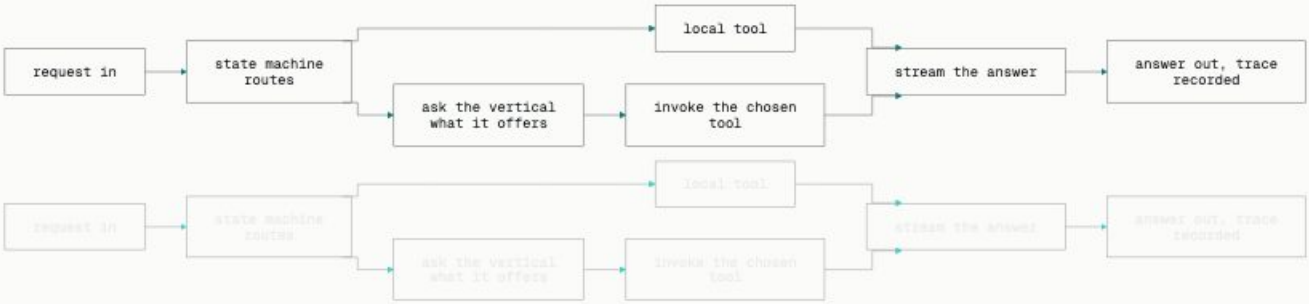


Figure 1

That middle step, asking instead of assuming, is the whole design. A vertical can ship a new tool without a new release of the hub, because the hub discovers capabilities instead of declaring them.

Where it meets the rest

The hub sits between the interfaces and the verticals. digichat and the dashboard send it chat-shaped requests; it answers in a widely compatible streaming shape, so ordinary chat clients work unchanged. Downstream it talks to digisearch for retrieval and digiquant for analysis, and to models through the shared model layer. Identity arrives as a short-lived credential minted upstream, which the hub verifies cryptographically instead of trusting the caller. Every step is traced through digitrace, so an operator can replay which tools ran, in what order, and how long each took.

Running it in each home

The hub runs the same in all three homes; only its neighbours change. On a laptop it sits beside the other services on the local network and talks to a local model runner. On Cloudflare it runs as part of the worker stack and reaches the verticals through service bindings. Self-hosted, it runs wherever the operator puts it and points at whichever model endpoints the operator chose. Nothing in the hub's own design prefers one home. That portability is why the hub never holds credentials longer than a single request.

> **Why digithings.** Most orchestration frameworks want to own your tools: you write to their plugin format and you are locked in. digigraph inverts that, tools are described over standard protocols and discovered at runtime, so the hub works with services it never heard of. That is the modular, no-lock-in position in one component.

06

digillm, one model layer for every service

Left alone, every service grows its own model client. Each one picks a favourite vendor, hard-codes the retry behaviour its author liked, and handles failures a little differently. Six months later, switching models means six refactors, and nobody can answer what a model call costs. digillm exists so that never happens: one library, every service, no vendor preference.

What it is for

The audience question is practical: what does it take to change models? With a shared layer, the answer is a configuration choice, not a rewrite. digillm speaks to any endpoint that follows the widely adopted chat format, so local runners, hosted vendors and routing proxies all look identical to the calling service. A team trying a new model changes where the traffic goes, not how the code asks.

How it works

digillm is a library, not a service: it ships inside the services that use it rather than running on its own. It defines the shared shapes every model call uses: the request, the tool call, the tool definition, and structured answers in schema form. On top of those shapes it layers the behaviours every caller needs but nobody wants to write twice: retries, timeouts, usage accounting, and optional tracing hooks. A routing proxy can sit underneath for provider choice, caching and rate limits, and the library never knows or cares.

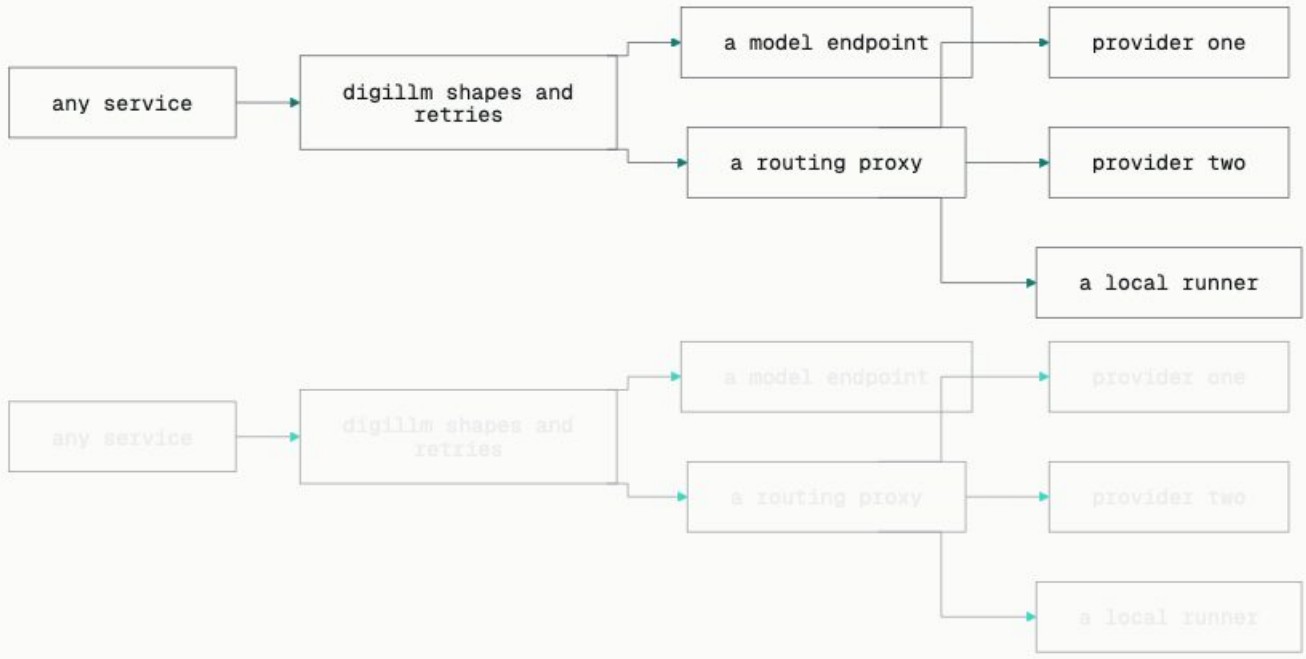


Figure 1

The dependency discipline is part of the design: the library holds almost no dependencies of its own and never imports web frameworks, so it can live inside a chat interface, an orchestration hub and a research pipeline without dragging their worlds along.

Where it meets the rest

digillm is consumed, not called. The orchestration hub uses it for routing-adjacent model calls, the search vertical for judging and rewriting, and the research tooling for structured extraction. Tracing flows through digitrace hooks when tracing is configured, and silently costs nothing when it is not. Because the shapes are shared, a structured answer produced in one service can be read in another without translation.

Running it in each home

A library has no deployment of its own, so the three homes change nothing about digillm itself, they change what sits underneath it. Locally it points at a model runner on the same machine. On Cloudflare it points at whatever inference the deployment chose. Self-hosted, the operator picks the endpoint and the library behaves identically. That indifference is the point: the decision of where models run stays an operator decision, never a code change.

> **Why digithings.** Swapping a model vendor is usually a migration project. Here it is a routing decision, because no service code names a vendor. Open interfaces plus one shared layer is what makes pay-as-you-go models genuinely interchangeable instead of merely available.

07

digichat, the interface people touch

Most people never meet the stack. They meet digichat: the chat surface where questions go in and answers stream out. Everything behind it, the hub, the verticals and the models, serves that surface. The surface stays thin on purpose.

What it is for

digichat answers one audience question: what does it feel like to use this system? A person signs in, opens a conversation, and talks. The interface streams answers as they arrive, keeps history per conversation, and offers artifacts, rendered outputs like documents and visuals, beside the chat. It also embeds elsewhere: a host site can frame the chat experience for its own visitors, with the same backend serving both.

How it works

digichat is two things that travel together: a request layer and a surface. The request layer authenticates the person, resolves which conversation they are in, and forwards the message to the orchestration hub with a short-lived identity. The surface renders the streaming answer, the history and the artifacts. Neither half holds anything an attacker would want: sessions are verified per request, conversation content lives in the data layer rather than in the interface, and embed visitors are authorised per host before they see anything.

The embed path follows the same shape with one extra check: the host naming the visitor must itself be recognised before any conversation resolves. A registered host alone authorises nothing; the per-visit credential does.

Where it meets the rest

digichat is the hub's most demanding caller and the data layer's steadiest writer. It leans on the identity service for sessions and machine credentials, on the hub for answers, and on the data layer for conversation persistence. Artifact rendering reuses the shared design components, so the chat looks like the rest of the surfaces without sharing their code.

Running it in each home

Locally the interface runs beside the stack it talks to, which makes development a single checkout. On Cloudflare it runs at the edge, close to visitors, while the hub and verticals sit behind service bindings. Self-hosted it runs wherever the operator serves web traffic from. In every home the interface stays stateless: stop it, move it, scale it, conversations survive because they never lived in it.

> **Why digithings.** The interface is open source and stateless, so it runs anywhere the operator already serves traffic. No hosted-only surface, no per-seat metering in the interface itself, the chat you run is the chat you own.

08

digiquant, the research vertical

Quantitative research has a failure mode other software does not: results that look rigorous and are not. A backtest that peeks at the future, a pipeline that reorders steps casually, a number nobody can reproduce, each one manufactures confidence. digiquant is the vertical that refuses those outcomes structurally, by making the honest order of operations the only order the pipeline can run.

What it is for

The audience question is trust: why should anyone believe a number this system produces? digiquant answers with process. Research flows through an ordered pipeline, data, features, strategy, backtest, optimisation, where each stage declares what it reads and the pipeline refuses to run stages out of order. Every run keeps its inputs, so a result can be re-examined later under the same conditions that produced it.

How it works

The pipeline is the unit, not the function. A caller asks for a research task; the vertical plans the stages, runs them through a mature backtesting engine, and returns the outcome with its provenance attached. Honesty checks sit inside the pipeline rather than beside it: estimates that are refused carry no number at all instead of a plausible-looking one, and partial results are labelled partial rather than rounded up. The orchestrator half of the vertical exposes the same tools the hub discovers at runtime, so the hub never needs to know how quant work is ordered, only what to ask for.

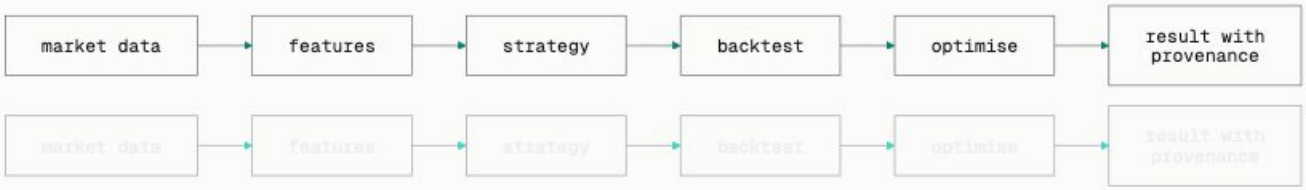


Figure 1

Each arrow is a contract: a stage may only read what the previous stage wrote. That single rule is what makes results reproducible and what makes look-ahead bias structurally hard instead of merely discouraged.

Where it meets the rest

digiquant is the hub's quant arm and the data layer's heaviest reader. Market history flows in from the data layer; research briefs and digests flow out to whoever asked, the chat surface, scheduled sessions, or the dashboard. Model calls go through the shared model layer, and every run is traced so a number on a screen can be walked back to the data and parameters behind it.

Running it in each home

The research pipeline runs wherever compute is cheapest for the operator. Locally it runs on the same machine as the data it reads, which keeps large backtests fast. On Cloudflare the scheduled runner executes on a timetable and the interactive parts stay behind the hub. Self-hosted, the operator sizes the machine to the workload. In every home the pipeline's ordering guarantees travel with it, they are in the code, not in the deployment.

> **Why digithings.** Institutional-grade research tooling usually arrives as a closed platform with the methodology hidden inside. Here the pipeline, the ordering rules and the honesty checks are open source and readable, so a small firm can hold its research to the same standard without buying the platform.

09

digivoice, the voice loop on your machine

Typing is not always the way people work. Sometimes the thought arrives spoken, half-formed, while walking or while the hands are busy. digivoice is the loop that catches it: capture speech, transcribe it locally, optionally reshape the transcript for its destination, and speak results back. The whole loop runs on the operator's own machine, with no audio ever leaving it.

What it is for

The audience question is about place: where should voice be processed? The common answer, send the audio to a hosted service, trades away the most sensitive stream a person produces: their voice, with all its background. digivoice answers that the machine in front of you is already fast enough. Local transcription and synthesis keep the loop private by construction, offline-capable, and free of per-minute metering.

How it works

The loop has four stations. Capture records from the microphone to a local audio file. Transcription turns that file into text with a local speech model. An optional rewrite pass reshapes the raw transcript for where it is going, a message, a note, a command, using a small local model with a cleanup prompt that keeps the boundary between instruction and content. Synthesis speaks results back through a local voice. History stays on the machine as an append-only log the operator can search and revisit.

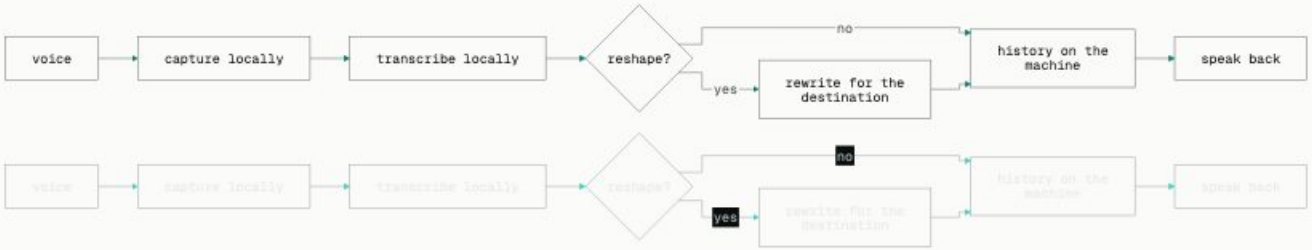


Figure 1

Settings are scriptable, get and set from the command line, so the loop can be driven by other tools, not only by fingers. A built-in checkup command reports what the host can and cannot do, because a local tool owes its operator an honest account of the host.

Where it meets the rest

digivoice touches the rest of the system lightly, by design. It shares the design language with the terminal surfaces, reads local model files in the same way the rest of the stack reads models, and can hand a finished transcript to the chat or the research pipeline as ordinary text. It never calls the hub mid-loop: the voice loop completes on the machine, and only its text output travels further.

Running it in each home

digivoice has one home: the operator's machine. It is not a service and it does not deploy to Cloudflare or to shared servers, voice capture needs the microphone in front of the person speaking. That constraint is also its strength: there is no voice infrastructure to operate, scale or pay for. The self-hosting story for voice is simply that it was never hosted at all.

> **Why digithings.** Voice tooling normally means streaming your speech to someone else's servers on someone else's meter. A local loop keeps the most personal input stream entirely on hardware you own, with no account, no metering and no network needed. Privacy as architecture, not as policy.

10

digisearch, finding what matters

Language models know a lot and remember nothing of yours. Everything an organisation knows, its documents, its history, its prior research, sits outside the model. digisearch is the bridge: it ingests the operator's documents, indexes them for meaning, and retrieves the right passages when a question needs them. Retrieval is load-bearing enough to deserve its own vertical, with its own pipeline and its own guarantees.

What it is for

The audience question is grounding: how do answers stay connected to real documents? Without retrieval, a model answers from training memory, fluent, confident, and occasionally invented. With it, the answer carries the passages it stands on, and a sceptical reader can check them. digisearch makes that the default shape of every answer that touches private knowledge.

How it works

The vertical keeps two paths strictly apart. Ingestion takes documents in, splits them into passages, and stores them with meaning-aware indexes so later questions can find them by content rather than by keyword. Answering takes a question, retrieves the passages most likely to matter, and hands them to the model alongside the question, the model reasons over supplied evidence instead of recalling. The two paths share storage but never logic: a document being indexed cannot change how a question is answered, and answering never writes to the index.

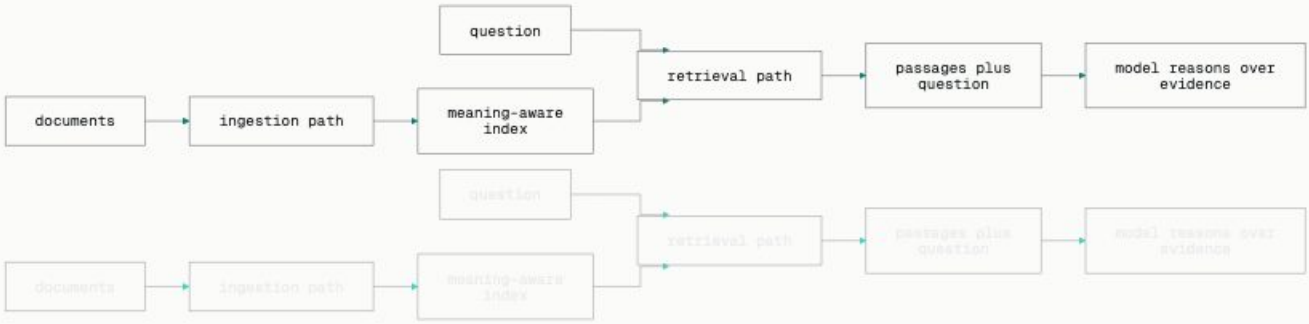


Figure 1

Access control rides along: retrieval respects which scopes the requester holds, so private documents stay private even though they share an index with public ones. The vector backend underneath is an operator choice, embedded for a laptop, managed for a fleet, and the pipeline above it does not care.

Where it meets the rest

digisearch is the hub's research arm alongside digiquant: where the quant vertical answers from market data, search answers from documents. The chat surface leans on it for grounded answers, and the research pipelines call it for background. Ingestion can be scheduled or on demand; either way it lands in the same index the answering path reads.

Running it in each home

The pipeline runs in every home; the index backend is what changes. Locally an embedded index keeps the whole loop on one machine. On Cloudflare the index lives in managed vector storage beside the workers. Self-hosted, the operator picks the backend that fits the corpus size. Documents cross no boundary the operator did not approve in any home, ingestion is always an explicit act.

> **Why digithings.** Grounded answers usually mean sending your documents to a hosted search vendor. Here ingestion, indexing and retrieval all run on infrastructure you control, over standard protocols, so private knowledge stays private while answers stay checkable.

11

The Cloudflare stack and its workers

Cloudflare is one of three homes, and the only one that runs on someone else's computers without becoming someone else's product. Workers handle request paths at the edge, containers run the workloads that need a full machine, and bindings connect them to state. The shape changes; the interface does not.

What it is for

The audience question is portability: if the system runs on a laptop today, what does it take to run it at the edge tomorrow? The answer the stack gives is: a deployment decision, not a rewrite. The same services, the same contracts, the same identity checks, placed on Cloudflare's network instead of a local machine. An operator outgrows a laptop without outgrowing the system.

How it works

Three Cloudflare shapes cover the stack. Workers serve the request-driven paths: the chat interface, the request layer, scheduled runners. Containers host the workloads that need persistent processes, model runners, research pipelines, tracing backends. Bindings connect the two to durable state and to each other, so a worker can reach a containerised service as if it were local. Identity and scopes work exactly as they do locally, because the services verify credentials themselves rather than trusting the network around them.

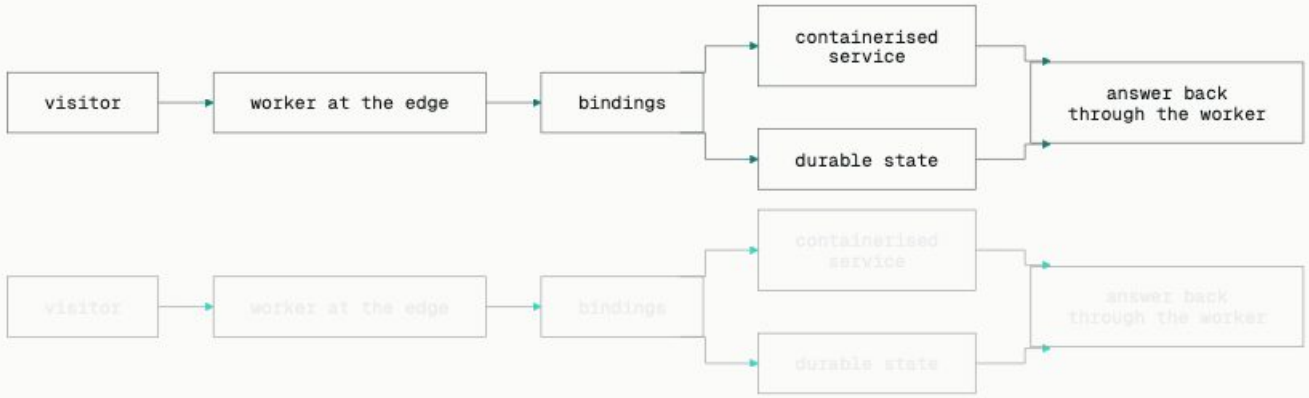


Figure 1

Nothing in this picture is Cloudflare-specific in design, only in address. The services do not know which home they are in, which is why the same build can move between homes without code changes.

Where it meets the rest

The edge home meets the operator through deployment, not through new concepts. Configuration declares which service lands where; the deploy pipeline builds and ships; scheduled runners keep their timetables. The local and self-hosted homes stay available for development and for operators who want every byte on their own hardware. Moving between homes is a supported direction in all directions, not a migration with no return.

> **Why digithings.** Edge deployment here does not mean re-platforming onto a vendor's framework. The system deploys to the edge the way it deploys anywhere: the same services, the same open protocols, usage-based billing with no hosted-tier features to envy. Reach without lock-in.

12

The local stack, one machine

Before the system runs anywhere else, it runs here: one machine, every service beside its neighbours, the full loop from question to answer on hardware the operator can touch. The local stack is not a demo mode or a crippled trial. It is the same system, composed for one machine, and for many operators it is the only home they ever need.

What it is for

The audience question is sufficiency: how much machine does this actually need? The local stack answers by being ordinary. Services start side by side, talk to each other over the machine's own network, and keep their data in local stores. A developer runs the same composition they ship; a small firm runs the whole product on one well-chosen box. Nobody needs permission, an account, or a card on file to begin.

How it works

Composition is declarative: one description says which services run, what each needs, and how they reach each other. Starting the stack brings them up in dependency order; stopping it puts everything away. The services behave identically to their siblings in other homes, same contracts, same identity checks, same tracing, because the code never learns which home it is in. The only local concession is convenience: friendlier defaults, warmer caches, and tooling that assumes a person is watching.

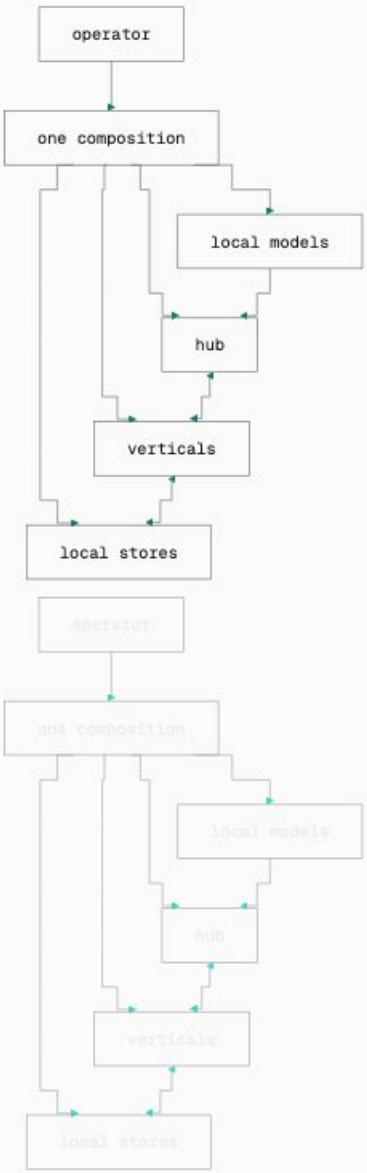


Figure 1

Seed content and example flows ship with the composition so the first run answers real questions immediately. From there the operator grows into the system by changing configuration, never by reinstalling it.

Where it meets the rest

The local home meets the operator's working day: development, experiment, demonstration, and honest small-scale production. When traffic or appetite outgrows one machine, the same services move to the edge or to self-hosted hardware with their contracts intact. Nothing written against the local stack needs rewriting, that guarantee is what makes the local home safe to invest in.

> **Why digithings.** A serious AI stack that starts on a laptop with no account and no meter is rare enough to be the point. The local home means evaluation costs nothing, development stays fast, and small operators never pay for scale they do not have.

13

MCP, every capability as a tool

A system with dozens of capabilities needs a way to say what each one does that both people and models can read. The industry answer is the Model Context Protocol: tools described with names, schemas and permissions, over a standard transport. digithings adopts it everywhere, every capability the system offers is a described tool, and every model-facing call goes through the same gate.

What it is for

The audience question is extension: how does something new plug in? The answer the protocol gives is uniformity. A new search source, a new analysis, a new integration, each arrives as a tool description, and every consumer in the system already knows how to read descriptions. Nobody patches the hub, the chat or the dashboard to adopt it. Describing is integrating.

How it works

Tools live behind servers that advertise them. The hub and the verticals list what a server offers, filter that list through the operator's allow-list, and expose only what survives to the model. Invocation carries the same identity the rest of the system uses, so a tool call is authorised with the same scopes as any request. A per-server gate decides which tools may run unattended and which need a person to approve, and that gate is checked where the tool executes, not only where it is offered, because offering and executing are different promises.

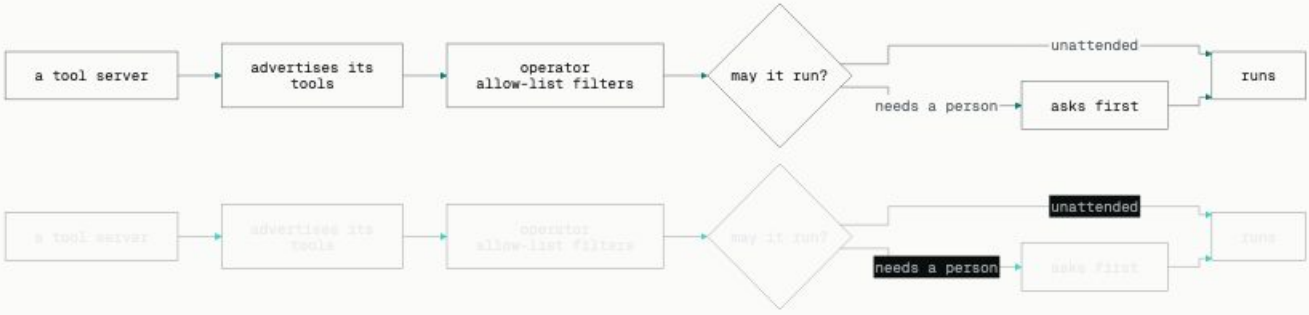


Figure 1

The subtle rule: a model is never offered a tool it could not invoke, and invocation never trusts the offer alone. Both halves are checked, every time, which is what keeps a clever prompt from reaching past its grant.

Where it meets the rest

The protocol is the system's connective tissue. Verticals publish their tools through it; the hub consumes them through it; outside agents and editors connect to the system's own tool servers through it. Operator-owned integrations, project trackers, team chat, ticketing, arrive as tool servers under the same gate, scoped as narrowly as the operator allows.

Running it in each home

The protocol is transport-agnostic, so homes change nothing about the contract. Locally, tool servers run as neighbours on the machine. On Cloudflare they sit behind the same bindings as everything else. Self-hosted they run wherever the operator keeps services. What changes per home is only which tool servers the operator connects, the gate, the scopes and the approval flow travel unchanged.

> **Why digithings.** Tool access is usually bespoke glue per integration. Standardising on the open protocol means integrations compose instead of accumulating, and the approval gate means a model with many tools is still governed. Modularity you can audit, not just admire.

14

Your machine first

Most infrastructure projects treat running locally as a favour to developers: a smaller, vaguer cousin of the real thing. digithings treats your machine as a first-class home. The stack runs there fully, hub, verticals, chat, tracing, and anything verified locally is expected to behave the same way in production, because both homes read the same contract.

The audience question this chapter answers is the sceptic's: why should I believe a demo on a laptop predicts production at all? Answer: because the two variants are not allowed to drift apart, and the checks that prove it run on every change.

What it is for

Self-hosting has two jobs. The first is independence: the software works without any hosted platform, so owning your infrastructure is a real option, not a slogan. The second is verification: interfaces, chat surfaces and tool integrations are proven fully connected locally before anything ships to the hosted platform. Local-first is the gate production walks through, not the waiting room beside it.

That only works if the two variants share one contract: the same setting names, the same service layout, the same secrets manifest. A variant supplies values; it never invents a name. Parity checks compare the two homes continuously, and drift fails loudly instead of accumulating quietly.

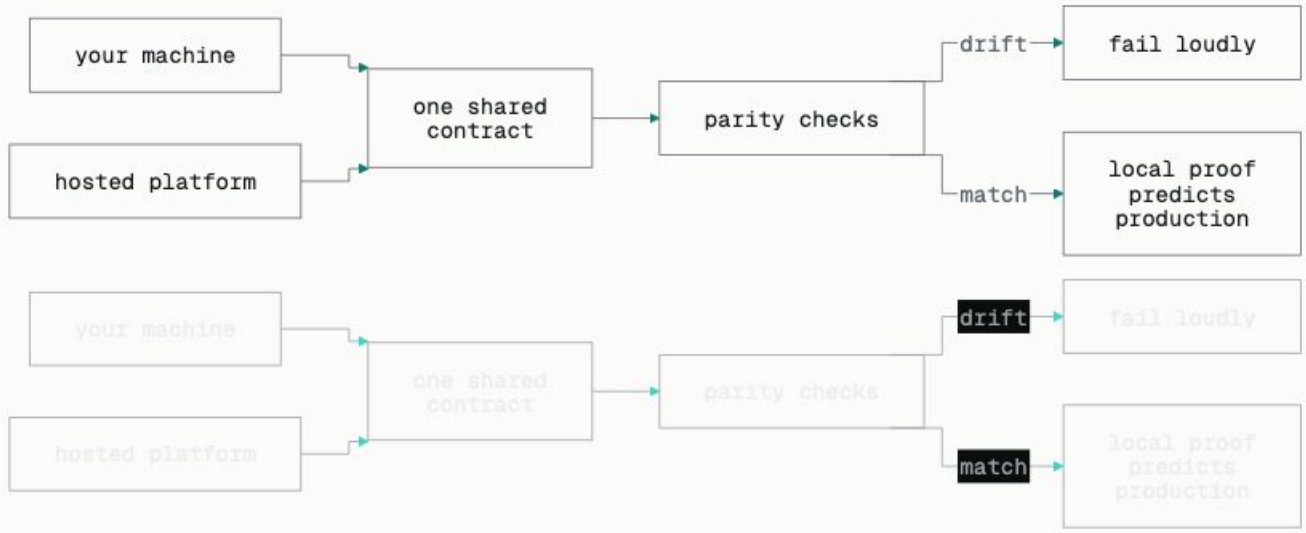


Figure 1

Modularity follows the same principle. Each capability opts in separately, so you run only what you need, the core set, the chat surface, the tracing backend, the tool integrations, and each addition is one explicit choice rather than a bundle you accept whole.

Where it meets the rest

The self-host variant touches everything the hosted one does: the same hub routes, the same model layer answers, the same data layer persists. The local model runner stands in for hosted inference; the local database stands in for the hosted one. Nothing in any module prefers one home, that neutrality is what the parity checks are actually testing.

Running it in each home

The home you own is the first-class one. The hosted platform is production only: there is no hosted development stack, because duplicating the whole infrastructure for development would double its cost. Development, review and verification happen on machines; production is where verified work lands. The asymmetry is deliberate and one-directional.

> **Why digithings.** Most platforms let you rent their cloud or leave. digithings proves, on every change, that its software runs without them, open source, reproducible steps, the same contract in both homes. That is the self-hosting position in one practice: independence you can verify.

DIGITHINGS

THE DIGITHINGS BOOK

edition 2026 · version 1.0